SESI TEORI

Dari Prosedural menuju OOP

Class, Object, Attribute, dan Method dasar di Python

Sub-CPMK 1 · Mata Kuliah Pemrograman Berorientasi Objek

CAPAIAN PEMBELAJARAN

Sub-CPMK 1: Apa yang Akan Dicapai?

Mampu **menjelaskan pergeseran paradigma** dari prosedural ke PBO, dan **mengimplementasikan Class, Object, Attribute, dan Method dasar** di Python.

1

Menjelaskan

Perbedaan mendasar cara berpikir prosedural vs berorientasi objek

2

Mengidentifikasi

Komponen Class: attribute (data) dan method (perilaku)

3

Membuat

Class dan Object sederhana menggunakan sintaks Python

4

Mengakses

Attribute dan memanggil Method dari sebuah Object

MENGINGAT KEMBALI

Apa itu Pemrograman Prosedural?

Program disusun sebagai **urutan instruksi (fungsi/prosedur)** yang dijalankan langkah demi langkah untuk memanipulasi data.

- Data dan fungsi terpisah
- Alur program bersifat linear / top-down
- Fungsi memanggil fungsi lain untuk mengolah data global

```python
# gaya prosedural
def hitung_luas(p, l):
    return p * l

def cetak_luas(p, l):
    luas = hitung_luas(p, l)
    print(f"Luas: {luas}")

cetak_luas(5, 3)
cetak_luas(8, 2)
```

Data (p, l) 'dikirim' ke fungsi — tidak menyatu dengan fungsinya

PERSOALAN

Tantangan Pendekatan Prosedural pada Sistem Besar

!

Data & Logika Terpisah

Sulit melacak fungsi mana saja yang mengubah suatu data ketika program membesar.

!

Duplikasi Kode

Struktur data serupa (mis. mahasiswa, pegawai) memicu fungsi berulang dengan pola yang mirip.

!

Sulit Dipelihara

Perubahan satu struktur data berisiko merusak banyak fungsi yang bergantung padanya.

!

Sulit Diskalakan

Menambah entitas baru berarti menambah banyak fungsi baru yang saling lepas.

PERGESERAN PARADIGMA

Evolusi Cara Berpikir dalam Menyusun Program



Prosedural

Fokus pada urutan instruksi & fungsi. Data terpisah dari proses.

Modular

Kode dikelompokkan menjadi modul/file agar lebih terorganisir.

Berorientasi Objek

Data (attribute) & perilaku (method) disatukan dalam Class dan Object.

OOP menyatukan apa yang dipisahkan oleh pendekatan prosedural.

KONSEP INTI

Apa itu Object-Oriented Programming (OOP)?

Paradigma pemrograman yang memodelkan program sebagai kumpulan “objek” — setiap objek membungkus data miliknya sendiri (attribute) beserta perilaku yang bisa dilakukannya (method).

“Objek = Data + Perilaku, menyatu dalam satu unit”

C **Class**

Cetakan/blueprint yang mendefinisikan struktur objek

O **Object**

Instance nyata yang dibuat dari sebuah Class

A **Attribute**

Data/karakteristik yang dimiliki objek

M **Method**

Fungsi yang menentukan perilaku objek

KONSEP CLASS

Class: Cetakan / Blueprint

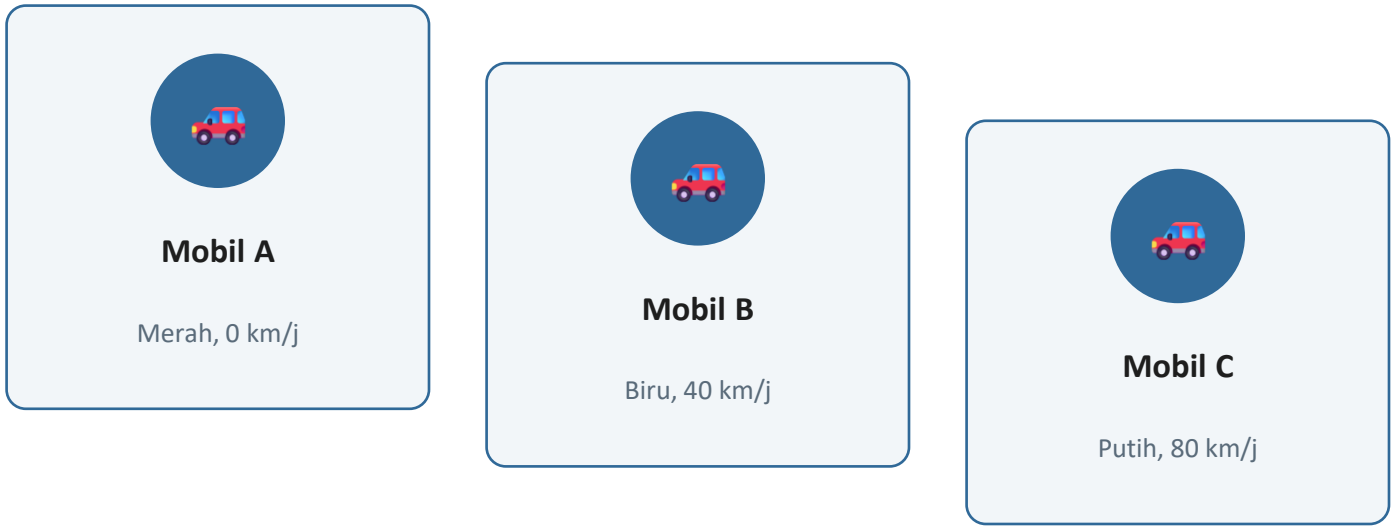
Class adalah rancangan/template yang mendefinisikan atribut dan method apa saja yang akan dimiliki objek-objek yang dibuat darinya. Class sendiri belum berupa objek nyata.

```
class Mobil
```

Attribute:
- merek
- warna
- kecepatan

Method:
- jalan()
- berhenti()

instansiasi



Satu Class dapat menghasilkan banyak Object dengan nilai attribute berbeda-beda.

KONSEP OBJECT

Object: Instance Nyata dari Class

Object adalah wujud nyata (instance) yang dibuat berdasarkan Class. Setiap object memiliki data attribute-nya sendiri, meski method-nya berasal dari Class yang sama.

1

Identity

Setiap object punya identitas unik di memori, walau atributnya sama persis dengan object lain

2

State

Kondisi object saat ini, ditentukan oleh nilai-nilai attribute-nya

3

Behavior

Apa yang bisa dilakukan object, ditentukan oleh method yang dimilikinya

KOMPONEN CLASS

Attribute: Data yang Dimiliki Objek

Attribute adalah variabel yang menyimpan karakteristik/keadaan sebuah object. Di Python, attribute biasanya dideklarasikan dengan awalan `self.` di dalam class.

Attribute	Contoh Nilai	Tipe
nama	"Andi"	str
nim	"2306001"	str
ipk	3.75	float

```
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama
        self.nim = nim
        self.ipk = 0.0

# nama, nim, ipk adalah ATTRIBUTE
# milik setiap object Mahasiswa
```

KOMPONEN CLASS

Method: Perilaku yang Dimiliki Objek

Method adalah fungsi yang didefinisikan di dalam class dan menentukan apa saja yang bisa dilakukan oleh object. Parameter pertama method selalu `self`, yang merujuk ke object itu sendiri.

- **`tampilkan_data()`**
  Menampilkan nama, nim, dan ipk mahasiswa
- **`update_ipk(nilai)`**
  Memperbarui nilai ipk milik object
- **`is_lulus()`**
  Mengembalikan status kelulusan berdasarkan ipk

```
class Mahasiswa:
    def __init__(self, nama, ipk):
        self.nama = nama
        self.ipk = ipk

    def tampilkan_data(self):
        print(self.nama, self.ipk)

    def is_lulus(self):
        return self.ipk >= 2.0

# tampilkan_data & is_lulus adalah METHOD
```

PERBANDINGAN

Prosedural vs OOP: Kasus yang Sama

PROSEDURAL

```
def buat_mhs(nama, ipk):
    return {"nama": nama, "ipk": ipk}

def tampilkan(mhs):
    print(mhs["nama"], mhs["ipk"])

a = buat_mhs("Andi", 3.5)
tampilkan(a)

# data & fungsi terpisah
```

OOP

```
class Mahasiswa:
    def __init__(self, nama, ipk):
        self.nama = nama
        self.ipk = ipk

    def tampilkan(self):
        print(self.nama, self.ipk)

a = Mahasiswa("Andi", 3.5)
a.tampilkan()
# data & fungsi menyatu di 'a'
```

SINTAKS PYTHON

Struktur Dasar Class di Python

```python
class NamaClass:
    def __init__(self, param1, param2):
        self.attr1 = param1
        self.attr2 = param2

    def nama_method(self):
        # gunakan self.attr1, self.attr2
        ...
```

`class`

Kata kunci untuk mendefinisikan sebuah Class baru

`__init__`

Constructor: dijalankan otomatis saat object dibuat

`self`

Merujuk ke object yang sedang aktif/dipakai

`self.attr`

Cara mendeklarasikan & mengakses attribute object

CONTOH LENGKAP

Dari Class ke Object: Contoh Utuh

```
class Mahasiswa:
    def __init__(self, nama, nim, ipk):
        self.nama = nama
        self.nim = nim
        self.ipk = ipk

    def tampilkan_data(self):
        print(f"{self.nama} ({self.nim})")
        print(f"IPK: {self.ipk}")

# --- membuat object ---
m1 = Mahasiswa("Sari", "2306010", 3.8)
m2 = Mahasiswa("Budi", "2306011", 3.2)

m1.tampilkan_data()
m2.tampilkan_data()
```

OUTPUT

```
Sari (2306010)
IPK: 3.8

Budi (2306011)
IPK: 3.2
```

m1 dan m2 adalah dua Object berbeda dari Class Mahasiswa yang sama — masing-masing punya nilai attribute sendiri.

RINGKASAN

Poin-Poin Penting Sesi Ini

1. Pemrograman prosedural memisahkan data dan fungsi, menyulitkan pemeliharaan program besar
2. OOP menyatukan data (attribute) dan perilaku (method) ke dalam satu unit bernama Object
3. Class adalah blueprint/cetakan; Object adalah instance nyata yang dibuat dari Class
4. Di Python, class dibuat dengan kata kunci class, constructor __init__, dan parameter self
5. Attribute diakses dengan self.nama_attribute; method dipanggil dengan object.nama_method()

REFERENSI & LANGKAH SELANJUTNYA

Lanjut ke Sesi Praktikum

- Sweigart, A. (2019). Automate the Boring Stuff with Python. No Starch Press.
- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist. O'Reilly.
- Python Software Foundation. Python Documentation — Classes. docs.python.org
- Lutz, M. (2013). Learning Python. O'Reilly Media.

Siapkan laptop dan Python 3 untuk sesi praktikum berikutnya.